

LeetCode Top 70 - Interview Priority Guide 2026

Sorted by Interview Frequency (FAANG+ 2024-2026)



PRINT SETTINGS

- **Mode:** Landscape
- **Font:** Courier New or Consolas, 9-10pt
- **Margins:** 0.5 inch all sides
- **Pages:** ~50-60 pages
- **Format:** 2-sided printing recommended



QUICK STATS

- **Total Problems:** 70 complete solutions
- **Tier 1 (Must Know):** Problems 1-12
- **Tier 2 (Very Important):** Problems 13-36
- **Tier 3 (Important):** Problems 37-70
- **Follow-up Q&As:** 35+ essential extensions



HOW TO USE THIS GUIDE

1. **Week 4-3 before interview:** Master Tier 1 (1-12)
2. **Week 2-1 before interview:** Complete Tier 2 (13-36)
3. **Last week:** Review Tier 3 + Follow-ups
4. **Day before:** Quick review of Pattern Recognition Guide



QUICK REFERENCE - TOP 12 MUST-KNOW

- | | |
|-------------------------|-----------------------------|
| 1. LRU Cache | 11. Longest Substring |
| 2. Course Schedule | 12. Kth Largest Element |
| 3. Number of Islands | |
| 4. Merge Intervals | If you only have 1 week: |
| 5. Top K Frequent | Focus on these 12 problems! |
| 6. Min Window Substring | |
| 7. Subarray Sum K | They cover 80% of patterns |
| 8. Search Rotated Array | and appear in 70%+ of |
| 9. Trapping Rain Water | FAANG interviews. |
| 10. 3Sum | |

PAGE BREAK



TIER 1: MUST KNOW (Top 12 - Learn These First!)

1. LC 146 - LRU Cache | 👍 22K ★★★★★

Pattern: HashMap + doubly linked list | **Time:** O(1) | **Space:** O(capacity)

Problem: Design LRU cache with O(1) get/put

Example: `LRUCache cache = new LRUCache(2); cache.put(1,1); cache.get(1) → 1`

```
class LRUCache {
    class Node {
        int key, value;
        Node prev, next;
        Node(int key, int value) { this.key = key; this.value = value; }
    }

    private Map<Integer, Node> cache = new HashMap<>();
    private Node head = new Node(0, 0), tail = new Node(0, 0);
    private int capacity;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public int get(int key) {
        if (!cache.containsKey(key)) return -1;
        Node node = cache.get(key);
        remove(node);
        addToHead(node);
        return node.value;
    }

    public void put(int key, int value) {
        if (cache.containsKey(key)) {
            Node node = cache.get(key);
            node.value = value;
            remove(node);
            addToHead(node);
        } else {
            if (cache.size() == capacity) {
                cache.remove(tail.prev.key);
                remove(tail.prev);
            }
            Node newNode = new Node(key, value);
            cache.put(key, newNode);
            addToHead(newNode);
        }
    }

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
}
```

```
private void addToHead(Node node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}
}
```

2. LC 207 - Course Schedule | 👍 18K ★★★★★

Pattern: Topological sort (Kahn's algorithm) | **Time:** $O(V+E)$ | **Space:** $O(V+E)$

Problem: Can you finish all courses given prerequisites? (cycle detection)

Example: `numCourses = 2, prerequisites = [[1,0]]` → `true`

```
public boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> graph = new ArrayList<>();
    int[] indegree = new int[numCourses];

    for (int i = 0; i < numCourses; i++) {
        graph.add(new ArrayList<>());
    }

    for (int[] prereq : prerequisites) {
        graph.get(prereq[1]).add(prereq[0]);
        indegree[prereq[0]]++;
    }

    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (indegree[i] == 0) queue.offer(i);
    }

    int completed = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        completed++;
        for (int next : graph.get(course)) {
            if (--indegree[next] == 0) {
                queue.offer(next);
            }
        }
    }
    return completed == numCourses;
}
```

3. LC 200 - Number of Islands | 🍌 25K ★★★★★

Pattern: DFS/BFS flood fill | **Time:** $O(m \times n)$ | **Space:** $O(m \times n)$

Problem: Count number of islands (1=land, 0=water)

Example: `grid = [{"1","1","0"}, {"1","1","0"}, {"0","0","1"}]` → 2

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(grid, i, j);
            }
        }
    }
    return count;
}

private void dfs(char[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length ||
        grid[i][j] == '0') {
        return;
    }
    grid[i][j] = '0';
    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
    dfs(grid, i, j + 1);
    dfs(grid, i, j - 1);
}
```

4. LC 56 - Merge Intervals | 🍌 24K ★★★★★

Pattern: Sort + merge overlapping | **Time:** $O(n \log n)$ | **Space:** $O(n)$

Problem: Merge all overlapping intervals

Example: `[[1,3],[2,6],[8,10],[15,18]]` → `[[1,6],[8,10],[15,18]]`

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    List<int[]> merged = new ArrayList<>();
    int[] currentInterval = intervals[0];
    merged.add(currentInterval);

    for (int[] interval : intervals) {
        if (interval[0] <= currentInterval[1]) {
            currentInterval[1] = Math.max(currentInterval[1], interval[1]);
        } else {
            merged.add(interval);
            currentInterval = interval;
        }
    }
    return merged.toArray(new int[merged.size()][2]);
}
```

```

        currentInterval[1] = Math.max(currentInterval[1], interval[1]);
    } else {
        currentInterval = interval;
        merged.add(currentInterval);
    }
}
return merged.toArray(new int[merged.size()][]);
}

```

5. LC 347 - Top K Frequent Elements | 👍 17K ★★★★★

Pattern: Bucket sort | **Time:** $O(n)$ | **Space:** $O(n)$

Problem: Return k most frequent elements

Example: `nums = [1,1,1,2,2,3]`, `k = 2` → `[1,2]`

```

public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freqMap = new HashMap<>();
    for (int num : nums) {
        freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
    }

    List<Integer>[] bucket = new List[nums.length + 1];
    for (int num : freqMap.keySet()) {
        int freq = freqMap.get(num);
        if (bucket[freq] == null) bucket[freq] = new ArrayList<>();
        bucket[freq].add(num);
    }

    int[] result = new int[k];
    int index = 0;
    for (int i = bucket.length - 1; i >= 0 && index < k; i--) {
        if (bucket[i] != null) {
            for (int num : bucket[i]) {
                result[index++] = num;
                if (index == k) return result;
            }
        }
    }
    return result;
}

```

6. LC 76 - Minimum Window Substring | 👍 20K ★★★★★

Pattern: Advanced sliding window | **Time:** $O(|s| + |t|)$ | **Space:** $O(|s| + |t|)$

Problem: Find minimum window in s containing all characters of t

Example: s = "ADOBECODEBANC", t = "ABC" → "BANC"

```
public String minWindow(String s, String t) {
    Map<Character, Integer> required = new HashMap<>();
    for (char c : t.toCharArray()) {
        required.put(c, required.getOrDefault(c, 0) + 1);
    }

    int requiredCount = required.size();
    int formed = 0;
    Map<Character, Integer> windowCounts = new HashMap<>();
    int left = 0, right = 0;
    int[] ans = {Integer.MAX_VALUE, 0, 0};

    while (right < s.length()) {
        char c = s.charAt(right);
        windowCounts.put(c, windowCounts.getOrDefault(c, 0) + 1);

        if (required.containsKey(c) &&
            windowCounts.get(c).intValue() == required.get(c).intValue()) {
            formed++;
        }

        while (left <= right && formed == requiredCount) {
            if (right - left + 1 < ans[0]) {
                ans[0] = right - left + 1;
                ans[1] = left;
                ans[2] = right;
            }

            char leftChar = s.charAt(left);
            windowCounts.put(leftChar, windowCounts.get(leftChar) - 1);
            if (required.containsKey(leftChar) &&
                windowCounts.get(leftChar) < required.get(leftChar)) {
                formed--;
            }
            left++;
        }
        right++;
    }

    return ans[0] == Integer.MAX_VALUE ? "" : s.substring(ans[1], ans[2] + 1);
}
```

7. LC 560 - Subarray Sum Equals K | 🍌 16K ★★★★★

Pattern: Prefix sum + HashMap | **Time:** O(n) | **Space:** O(n)

Problem: Count subarrays with sum = k

Example: `nums = [1,1,1]`, `k = 2` → 2

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixSumCount = new HashMap<>();
    prefixSumCount.put(0, 1);
    int count = 0, currentSum = 0;

    for (int num : nums) {
        currentSum += num;
        if (prefixSumCount.containsKey(currentSum - k)) {
            count += prefixSumCount.get(currentSum - k);
        }
        prefixSumCount.put(currentSum,
            prefixSumCount.getOrDefault(currentSum, 0) + 1);
    }
    return count;
}
```

8. LC 33 - Search in Rotated Sorted Array | 🍌 30K ★★★★★

Pattern: Modified binary search | **Time:** $O(\log n)$ | **Space:** $O(1)$

Problem: Search target in rotated sorted array

Example: `nums = [4,5,6,7,0,1,2]`, `target = 0` → 4

```
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;

        if (nums[left] <= nums[mid]) { // Left half is sorted
            if (target >= nums[left] && target < nums[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        } else { // Right half is sorted
            if (target > nums[mid] && target <= nums[right]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
    }
}
```

```
    return -1;
}
```

9. LC 42 - Trapping Rain Water | 👍 36K ★★★★★

Pattern: Two pointers with max tracking | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Calculate trapped rainwater

Example: `height = [0,1,0,2,1,0,1,3,2,1,2,1]` → 6

```
public int trap(int[] height) {
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;

    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax) {
                leftMax = height[left];
            } else {
                water += leftMax - height[left];
            }
            left++;
        } else {
            if (height[right] >= rightMax) {
                rightMax = height[right];
            } else {
                water += rightMax - height[right];
            }
            right--;
        }
    }
    return water;
}
```

10. LC 15 - 3Sum | 👍 35K ★★★★★

Pattern: Sort + two pointers | **Time:** $O(n^2)$ | **Space:** $O(1)$

Problem: Find all triplets that sum to zero

Example: `nums = [-1,0,1,2,-1,-4]` → `[[-1,-1,2], [-1,0,1]]`

```
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    Arrays.sort(nums);
```

```

    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left],
nums[right]));
                while (left < right && nums[left] == nums[left + 1])
                    left++;
                while (left < right && nums[right] == nums[right - 1])
                    right--;
                left++;
                right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }
    return result;
}

```

11. LC 3 - Longest Substring Without Repeating | 🍌 44K ★★★★★

Pattern: Sliding window + HashMap | **Time:** O(n) | **Space:** O(min(n, charset))

Problem: Find length of longest substring without repeating characters

Example: s = "abcabcbb" → 3 (substring: "abc")

```

public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int maxLength = 0;
    int left = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= left) {
            left = lastSeen.get(c) + 1;
        }
        lastSeen.put(c, right);
        maxLength = Math.max(maxLength, right - left + 1);
    }
    return maxLength;
}

```

12. LC 215 - Kth Largest Element | 👍 18K ★★★★★

Pattern: Min heap of size k | **Time:** $O(n \log k)$ | **Space:** $O(k)$

Problem: Find kth largest element

Example: `nums = [3,2,1,5,6,4]`, `k = 2` → 5

```
public int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();
    for (int num : nums) {
        minHeap.offer(num);
        if (minHeap.size() > k) {
            minHeap.poll();
        }
    }
    return minHeap.peek();
}
```

★★★★★ TIER 2: VERY IMPORTANT (13-36)

13. LC 23 - Merge K Sorted Lists | 👍 21K ★★★★★

Pattern: Min heap | **Time:** $O(N \log k)$ | **Space:** $O(k)$

Problem: Merge k sorted linked lists

Example: `lists = [[1,4,5],[1,3,4],[2,6]]` → `[1,1,2,3,4,4,5,6]`

```
public ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap = new PriorityQueue<>((a,b) -> a.val - b.val);
    for (ListNode node : lists) {
        if (node != null) heap.offer(node);
    }

    ListNode dummy = new ListNode(0);
    ListNode tail = dummy;

    while (!heap.isEmpty()) {
        ListNode node = heap.poll();
        tail.next = node;
        tail = tail.next;
        if (node.next != null) {
            heap.offer(node.next);
        }
    }
}
```

```
    return dummy.next;
}
```

14. LC 124 - Binary Tree Maximum Path Sum | 👍 16K ★★★★★

Pattern: Postorder DFS | **Time:** $O(n)$ | **Space:** $O(h)$

Problem: Find maximum path sum (path can start/end anywhere)

Example: `root = [-10,9,20,null,null,15,7]` → 42 (15→20→7)

```
int maxSum = Integer.MIN_VALUE;

public int maxPathSum(TreeNode root) {
    maxGain(root);
    return maxSum;
}

private int maxGain(TreeNode node) {
    if (node == null) return 0;
    int leftGain = Math.max(maxGain(node.left), 0);
    int rightGain = Math.max(maxGain(node.right), 0);

    maxSum = Math.max(maxSum, node.val + leftGain + rightGain);
    return node.val + Math.max(leftGain, rightGain);
}
```

15. LC 236 - Lowest Common Ancestor | 👍 17K ★★★★★

Pattern: Recursive postorder | **Time:** $O(n)$ | **Space:** $O(h)$

Problem: Find LCA of two nodes

Example: `root = [3,5,1,6,2,0,8]`, `p = 5`, `q = 1` → 3

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q)
{
    if (root == null || root == p || root == q) return root;

    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);

    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

16. LC 322 - Coin Change | 👍 20K ★★★★★

Pattern: Unbounded knapsack | **Time:** $O(\text{amount} \times \text{coins})$ | **Space:** $O(\text{amount})$

Problem: Minimum coins to make amount

Example: `coins = [1,2,5]`, `amount = 11` → 3 (5+5+1)

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

17. LC 139 - Word Break | 👍 17K ★★★★★

Pattern: DP substring | **Time:** $O(n^3)$ | **Space:** $O(n)$

Problem: Can s be segmented into dictionary words?

Example: `s = "leetcode"`, `wordDict = ["leet","code"]` → true

```
public boolean wordBreak(String s, List<String> wordDict) {
    Set<String> wordSet = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length() + 1];
    dp[0] = true;

    for (int i = 1; i <= s.length(); i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && wordSet.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[s.length()];
}
```

18. LC 300 - Longest Increasing Subsequence | 👍 17K ★★★★★

Pattern: Binary search DP | **Time:** $O(n \log n)$ | **Space:** $O(n)$

Problem: Find length of longest strictly increasing subsequence

Example: `nums = [10,9,2,5,3,7,101,18]` → 4 ([2,3,7,101])

```
public int lengthOfLIS(int[] nums) {
    List<Integer> tails = new ArrayList<>();
    for (int num : nums) {
        int pos = Collections.binarySearch(tails, num);
        if (pos < 0) pos = -(pos + 1);
        if (pos == tails.size()) {
            tails.add(num);
        } else {
            tails.set(pos, num);
        }
    }
    return tails.size();
}
```

19. LC 127 - Word Ladder | 👍 15K ★★★★★

Pattern: BFS shortest path | **Time:** $O(M^2 \times N)$ | **Space:** $O(M \times N)$

Problem: Find shortest transformation sequence length

Example: `beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","cog"]`
→ 5

```
public int ladderLength(String beginWord, String endWord, List<String> wordList) {
    Set<String> wordSet = new HashSet<>(wordList);
    if (!wordSet.contains(endWord)) return 0;

    Queue<String> queue = new LinkedList<>();
    queue.offer(beginWord);
    int level = 1;

    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            String word = queue.poll();
            char[] chars = word.toCharArray();

            for (int j = 0; j < chars.length; j++) {
```

```

        char original = chars[j];
        for (char c = 'a'; c <= 'z'; c++) {
            if (c == original) continue;
            chars[j] = c;
            String newWord = new String(chars);

            if (newWord.equals(endWord)) return level + 1;
            if (wordSet.contains(newWord)) {
                queue.offer(newWord);
                wordSet.remove(newWord);
            }
        }
        chars[j] = original;
    }
    level++;
}
return 0;
}

```

20. LC 128 - Longest Consecutive Sequence | 👍 19K ★★★★★

Pattern: HashSet with smart iteration | **Time:** O(n) | **Space:** O(n)

Problem: Find length of longest consecutive sequence

Example: `nums = [100,4,200,1,3,2]` → 4 (sequence: [1,2,3,4])

```

public int longestConsecutive(int[] nums) {
    Set<Integer> numSet = new HashSet<>();
    for (int num : nums) numSet.add(num);

    int longestStreak = 0;
    for (int num : numSet) {
        if (!numSet.contains(num - 1)) { // Only start from sequence
beginning
            int currentNum = num;
            int currentStreak = 1;
            while (numSet.contains(currentNum + 1)) {
                currentNum++;
                currentStreak++;
            }
            longestStreak = Math.max(longestStreak, currentStreak);
        }
    }
    return longestStreak;
}

```

21. LC 238 - Product of Array Except Self | 🍌 24K ★★★★★

Pattern: Prefix/Suffix products | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Return array where $\text{answer}[i] = \text{product of all except nums}[i]$

Example: $\text{nums} = [1, 2, 3, 4] \rightarrow [24, 12, 8, 6]$

```
public int[] productExceptSelf(int[] nums) {
    int n = nums.length;
    int[] answer = new int[n];

    answer[0] = 1;
    for (int i = 1; i < n; i++) {
        answer[i] = answer[i - 1] * nums[i - 1];
    }

    int suffixProduct = 1;
    for (int i = n - 1; i >= 0; i--) {
        answer[i] *= suffixProduct;
        suffixProduct *= nums[i];
    }
    return answer;
}
```

22. LC 49 - Group Anagrams | 🍌 22K ★★★★★

Pattern: Sorted string as key | **Time:** $O(n \times k \log k)$ | **Space:** $O(n \times k)$

Problem: Group strings that are anagrams

Example: $["eat", "tea", "tan", "ate", "nat", "bat"] \rightarrow [["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]$

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String str : strs) {
        char[] chars = str.toCharArray();
        Arrays.sort(chars);
        String key = new String(chars);
        map.putIfAbsent(key, new ArrayList<>());
        map.get(key).add(str);
    }
    return new ArrayList<>(map.values());
}
```

23. LC 53 - Maximum Subarray (Kadane's) | 🍌 37K ★★★★★

Pattern: Kadane's algorithm | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Find contiguous subarray with largest sum

Example: `nums = [-2,1,-3,4,-1,2,1,-5,4]` → 6 ([4,-1,2,1])

```
public int maxSubArray(int[] nums) {
    int currentSum = nums[0], maxSum = nums[0];
    for (int i = 1; i < nums.length; i++) {
        currentSum = Math.max(nums[i], currentSum + nums[i]);
        maxSum = Math.max(maxSum, currentSum);
    }
    return maxSum;
}
```

24. LC 84 - Largest Rectangle in Histogram | 🍌 19K ★★★★★

Pattern: Monotonic stack | **Time:** $O(n)$ | **Space:** $O(n)$

Problem: Find largest rectangle in histogram

Example: `heights = [2,1,5,6,2,3]` → 10

```
public int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int maxArea = 0;

    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 : heights[i];
        while (!stack.isEmpty() && heights[stack.peek()] > h) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}
```

25. LC 79 - Word Search | 🍌 18K ★★★★★

Pattern: Backtracking DFS | **Time:** $O(m \times n \times 4^L)$ | **Space:** $O(L)$

Problem: Find if word exists in grid

Example: `board = [ ["A","B","C"], ["S","F","C"], ["A","D","E"] ], word = "ABCCED" → true`

```
public boolean exist(char[][] board, String word) {
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            if (dfs(board, word, 0, i, j)) return true;
        }
    }
    return false;
}

private boolean dfs(char[][] board, String word, int index, int i, int j) {
    if (index == word.length()) return true;
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length ||
        board[i][j] != word.charAt(index)) return false;

    char temp = board[i][j];
    board[i][j] = '#';

    boolean found = dfs(board, word, index+1, i+1, j) ||
                    dfs(board, word, index+1, i-1, j) ||
                    dfs(board, word, index+1, i, j+1) ||
                    dfs(board, word, index+1, i, j-1);

    board[i][j] = temp;
    return found;
}
```

26. LC 287 - Find Duplicate Number | 🍌 16K ★★★★★

Pattern: Floyd's cycle detection | **Time:** O(n) | **Space:** O(1)

Problem: Find the duplicate number (array has n+1 integers in range [1,n])

Example: `nums = [1,3,4,2,2] → 2`

```
public int findDuplicate(int[] nums) {
    int slow = nums[0];
    int fast = nums[0];

    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

```
        fast = nums[fast];
    }
    return slow;
}
```

27. LC 25 - Reverse Nodes in K-Group | 👍 16K ★★★★★

Pattern: K-group reversal | **Time:** O(n) | **Space:** O(1)

Problem: Reverse nodes in groups of k

Example: head = [1,2,3,4,5], k = 2 → [2,1,4,3,5]

```
public ListNode reverseKGroup(ListNode head, int k) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode prevGroupEnd = dummy;

    while (true) {
        ListNode kthNode = getKth(prevGroupEnd, k);
        if (kthNode == null) break;

        ListNode groupStart = prevGroupEnd.next;
        ListNode nextGroupStart = kthNode.next;

        // Reverse group
        ListNode prev = nextGroupStart;
        ListNode curr = groupStart;
        while (curr != nextGroupStart) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }

        prevGroupEnd.next = kthNode;
        prevGroupEnd = groupStart;
    }
    return dummy.next;
}

private ListNode getKth(ListNode curr, int k) {
    while (curr != null && k > 0) {
        curr = curr.next;
        k--;
    }
    return curr;
}
```

28. LC 206 - Reverse Linked List | 👍 26K ★★★★★

Pattern: Iterative three pointers | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Reverse a linked list

Example: $\text{head} = [1, 2, 3, 4, 5] \rightarrow [5, 4, 3, 2, 1]$

```
public ListNode reverseList(ListNode head) {  
    ListNode prev = null, curr = head;  
    while (curr != null) {  
        ListNode next = curr.next;  
        curr.next = prev;  
        prev = curr;  
        curr = next;  
    }  
    return prev;  
}
```

29. LC 2 - Add Two Numbers | 👍 36K ★★★★★

Pattern: Linked list traversal with carry | **Time:** $O(\max(m, n))$ | **Space:** $O(\max(m, n))$

Problem: Add two numbers represented as linked lists

Example: $l1 = [2, 4, 3], l2 = [5, 6, 4] \rightarrow [7, 0, 8]$ ($342 + 465 = 807$)

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {  
    ListNode dummy = new ListNode(0);  
    ListNode curr = dummy;  
    int carry = 0;  
  
    while (l1 != null || l2 != null || carry > 0) {  
        int sum = carry;  
        if (l1 != null) { sum += l1.val; l1 = l1.next; }  
        if (l2 != null) { sum += l2.val; l2 = l2.next; }  
  
        carry = sum / 10;  
        curr.next = new ListNode(sum % 10);  
        curr = curr.next;  
    }  
    return dummy.next;  
}
```

30. LC 55 - Jump Game | 👍 22K ★★★★★

Pattern: Greedy | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Can you reach the last index?

Example: `nums = [2,3,1,1,4]` → `true`

```
public boolean canJump(int[] nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > maxReach) return false;
        maxReach = Math.max(maxReach, i + nums[i]);
        if (maxReach >= nums.length - 1) return true;
    }
    return true;
}
```

31. LC 45 - Jump Game II | 👍 16K ★★★★★

Pattern: Greedy BFS | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Minimum jumps to reach last index

Example: `nums = [2,3,1,1,4]` → `2`

```
public int jump(int[] nums) {
    int jumps = 0, currentEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
        if (i == currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}
```

32. LC 152 - Maximum Product Subarray | 👍 16K ★★★★★

Pattern: Track max and min | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Find contiguous subarray with largest product

Example: `nums = [2,3,-2,4]` → `6` ([2,3])

```
public int maxProduct(int[] nums) {
    int maxProd = nums[0], currentMax = nums[0], currentMin = nums[0];
```

```
    for (int i = 1; i < nums.length; i++) {
        int temp = Math.max(nums[i], Math.max(currentMax * nums[i],
currentMin * nums[i]));
        currentMin = Math.min(nums[i], Math.min(currentMax * nums[i],
currentMin * nums[i]));
        currentMax = temp;
        maxProd = Math.max(maxProd, currentMax);
    }
    return maxProd;
}
```

33. LC 198 - House Robber | 🍌 20K ★★★★★

Pattern: DP with skip pattern | **Time:** O(n) | **Space:** O(1)

Problem: Max money without robbing adjacent houses

Example: `nums = [2,7,9,3,1]` → 12 (2+9+1)

```
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];
    int prev2 = nums[0], prev1 = Math.max(nums[0], nums[1]);
    for (int i = 2; i < nums.length; i++) {
        int current = Math.max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

34. LC 46 - Permutations | 🍌 21K ★★★★★

Pattern: Backtracking with visited | **Time:** O(n!) | **Space:** O(n)

Problem: Generate all permutations

Example: `nums = [1,2,3]` → `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), nums, new boolean[nums.length]);
    return result;
}

private void backtrack(List<List<Integer>> result, List<Integer> temp,
    int[] nums, boolean[] used) {
    if (temp.size() == nums.length) {
        result.add(new ArrayList<>(temp));
    }
    for (int i = 0; i < nums.length; i++) {
        if (!used[i]) {
            temp.add(nums[i]);
            used[i] = true;
            backtrack(result, temp, nums, used);
            temp.remove(temp.size() - 1);
            used[i] = false;
        }
    }
}
```

```

        if (temp.size() == nums.length) {
            result.add(new ArrayList<>(temp));
            return;
        }
        for (int i = 0; i < nums.length; i++) {
            if (used[i]) continue;
            temp.add(nums[i]);
            used[i] = true;
            backtrack(result, temp, nums, used);
            used[i] = false;
            temp.remove(temp.size() - 1);
        }
    }
}

```

35. LC 78 - Subsets | 🍌 19K ★★★★★

Pattern: Backtracking | **Time:** $O(2^n)$ | **Space:** $O(n)$

Problem: Generate all subsets

Example: `nums = [1,2,3]` → `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

```

public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), nums, 0);
    return result;
}

private void backtrack(List<List<Integer>> result, List<Integer> temp,
                        int[] nums, int start) {
    result.add(new ArrayList<>(temp));
    for (int i = start; i < nums.length; i++) {
        temp.add(nums[i]);
        backtrack(result, temp, nums, i + 1);
        temp.remove(temp.size() - 1);
    }
}

```

36. LC 739 - Daily Temperatures | 🍌 17K ★★★★★

Pattern: Monotonic stack | **Time:** $O(n)$ | **Space:** $O(n)$

Problem: Days until warmer temperature

Example: `temperatures = [73,74,75,71,69,72,76,73]` → `[1,1,4,2,1,1,0,0]`

```
public int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] answer = new int[n];
    Stack<Integer> stack = new Stack<>();

    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] >
temperatures[stack.peek()]) {
            int prevIndex = stack.pop();
            answer[prevIndex] = i - prevIndex;
        }
        stack.push(i);
    }
    return answer;
}
```

☆☆☆ TIER 3: IMPORTANT (37-70)

37. LC 1 - Two Sum | 👍 40K ☆☆☆☆☆

Pattern: HashMap for complement lookup | **Time:** O(n) | **Space:** O(n)

Problem: Find two indices where $\text{nums}[i] + \text{nums}[j] = \text{target}$

Example: $\text{nums} = [2, 7, 11, 15]$, $\text{target} = 9 \rightarrow [0, 1]$

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[] {map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[] {};
}
```

38. LC 121 - Best Time to Buy and Sell Stock | 👍 20K ☆☆☆☆☆

Pattern: Track min price | **Time:** O(n) | **Space:** O(1)

Problem: Find maximum profit from one buy and one sell

Example: $\text{prices} = [7, 1, 5, 3, 6, 4] \rightarrow 5$ (buy at 1, sell at 6)

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE;
    int maxProfit = 0;
    for (int price : prices) {
        minPrice = Math.min(minPrice, price);
        maxProfit = Math.max(maxProfit, price - minPrice);
    }
    return maxProfit;
}
```

39. LC 217 - Contains Duplicate | 👍 15K ★★★★★

Pattern: HashSet | **Time:** $O(n)$ | **Space:** $O(n)$

Problem: Return true if any value appears twice

Example: `nums = [1,2,3,1]` → `true`

```
public boolean containsDuplicate(int[] nums) {
    Set<Integer> seen = new HashSet<>();
    for (int num : nums) {
        if (!seen.add(num)) return true;
    }
    return false;
}
```

40. LC 189 - Rotate Array | 👍 12K ★★★★★

Pattern: Reverse method | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Rotate array to the right by k steps

Example: `nums = [1,2,3,4,5,6,7]`, `k = 3` → `[5,6,7,1,2,3,4]`

```
public void rotate(int[] nums, int k) {
    k = k % nums.length;
    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}

private void reverse(int[] nums, int start, int end) {
    while (start < end) {
        int temp = nums[start];
        nums[start++] = nums[end];
        nums[end--] = temp;
    }
}
```

```
        nums[end--] = temp;
    }
}
```

41. LC 57 - Insert Interval | 👍 15K ★★★★★

Pattern: Three-phase insertion | **Time:** $O(n)$ | **Space:** $O(n)$

Problem: Insert newInterval and merge if necessary

Example: `intervals = [[1,3],[6,9]], newInterval = [2,5] → [[1,5],[6,9]]`

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0;

    // Add intervals before newInterval
    while (i < intervals.length && intervals[i][1] < newInterval[0]) {
        result.add(intervals[i++]);
    }

    // Merge overlapping intervals
    while (i < intervals.length && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    result.add(newInterval);

    // Add remaining intervals
    while (i < intervals.length) {
        result.add(intervals[i++]);
    }

    return result.toArray(new int[result.size()][2]);
}
```

42. LC 11 - Container With Most Water | 👍 20K ★★★★★

Pattern: Two pointers from ends | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Find max area between two lines

Example: `height = [1,8,6,2,5,4,8,3,7] → 49`

```
public int maxArea(int[] height) {
    int left = 0, right = height.length - 1;
    int maxArea = 0;

    while (left < right) {
        int area = Math.min(height[left], height[right]) * (right - left);
        maxArea = Math.max(maxArea, area);
        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }
    return maxArea;
}
```

43. LC 125 - Valid Palindrome | 👍 10K ★★★★★

Pattern: Two pointers | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Check if string is palindrome (alphanumeric only)

Example: $s = \text{"A man, a plan, a canal: Panama"} \rightarrow \text{true}$

```
public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left)))
            left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right)))
            right--;
        if (Character.toLowerCase(s.charAt(left)) !=
            Character.toLowerCase(s.charAt(right))) return false;
        left++;
        right--;
    }
    return true;
}
```

44. LC 283 - Move Zeroes | 👍 15K ★★★★★

Pattern: Two pointers in-place | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Move all 0's to end while maintaining order

Example: $\text{nums} = [0, 1, 0, 3, 12] \rightarrow [1, 3, 12, 0, 0]$

```
public void moveZeroes(int[] nums) {
    int writePos = 0;
    for (int readPos = 0; readPos < nums.length; readPos++) {
        if (nums[readPos] != 0) {
            nums[writePos++] = nums[readPos];
        }
    }
    while (writePos < nums.length) {
        nums[writePos++] = 0;
    }
}
```

45. LC 443 - String Compression | 🍌 8K ⭐⭐⭐

Pattern: Two pointers write pattern | **Time:** O(n) | **Space:** O(1)

Problem: Compress characters in-place

Example: chars = ["a","a","b","b","c","c","c"] → 6 (becomes ["a","2","b","2","c","3"])

```
public int compress(char[] chars) {
    int write = 0, read = 0;
    while (read < chars.length) {
        char current = chars[read];
        int count = 0;
        while (read < chars.length && chars[read] == current) {
            read++;
            count++;
        }
        chars[write++] = current;
        if (count > 1) {
            for (char c : String.valueOf(count).toCharArray()) {
                chars[write++] = c;
            }
        }
    }
    return write;
}
```

46. LC 1248 - Count Subarrays with K Odd Numbers | 🍌 5K ⭐⭐⭐⭐

Pattern: atMost(k) - atMost(k-1) technique | **Time:** O(n) | **Space:** O(1)

Problem: Count subarrays with exactly k odd numbers

Example: nums = [1,1,2,1,1], k = 3 → 2 (subarrays: [1,1,2,1,1], [1,2,1,1])

```

public int numberOfSubarrays(int[] nums, int k) {
    return atMost(nums, k) - atMost(nums, k - 1);
}

private int atMost(int[] nums, int k) {
    int count = 0, left = 0, oddCount = 0;
    for (int right = 0; right < nums.length; right++) {
        if (nums[right] % 2 == 1) oddCount++;

        while (oddCount > k) {
            if (nums[left] % 2 == 1) oddCount--;
            left++;
        }

        count += right - left + 1; // All subarrays ending at right
    }
    return count;
}

```

Key Insight: $\text{exactly}(k) = \text{atMost}(k) - \text{atMost}(k-1)$. This technique works for "exactly k" problems.

47. LC 239 - Sliding Window Maximum | 🍌 16K ★★★★★

Pattern: Monotonic deque | **Time:** $O(n)$ | **Space:** $O(k)$

Problem: Return max element in each window of size k

Example: $\text{nums} = [1, 3, -1, -3, 5, 3, 6, 7]$, $k = 3 \rightarrow [3, 3, 5, 5, 6, 7]$

```

public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] result = new int[n - k + 1];
    Deque<Integer> deque = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        while (!deque.isEmpty() && deque.peekFirst() < i - k + 1) {
            deque.pollFirst();
        }
        while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
            deque.pollLast();
        }
        deque.offerLast(i);
        if (i >= k - 1) {
            result[i - k + 1] = nums[deque.peekFirst()];
        }
    }
}

```

```
    return result;
}
```

48. LC 643 - Maximum Average Subarray I | 🍌 8K ⭐⭐⭐

Pattern: Fixed sliding window | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Find max average of subarray of length k

Example: `nums = [1,12,-5,-6,50,3]`, `k = 4` → 12.75

```
public double findMaxAverage(int[] nums, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];
    int maxSum = sum;

    for (int i = k; i < nums.length; i++) {
        sum = sum - nums[i - k] + nums[i];
        maxSum = Math.max(maxSum, sum);
    }
    return (double) maxSum / k;
}
```

49. LC 1004 - Max Consecutive Ones III | 🍌 7K ⭐⭐⭐⭐

Pattern: Sliding window with constraint | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Max consecutive 1's if you can flip at most k 0's

Example: `nums = [1,1,1,0,0,0,1,1,1,1,0]`, `k = 2` → 6

```
public int longestOnes(int[] nums, int k) {
    int left = 0, maxLen = 0, zeros = 0;
    for (int right = 0; right < nums.length; right++) {
        if (nums[right] == 0) zeros++;
        while (zeros > k) {
            if (nums[left] == 0) zeros--;
            left++;
        }
        maxLen = Math.max(maxLen, right - left + 1);
    }
    return maxLen;
}
```

50. LC 1456 - Maximum Vowels in Substring | 🍌 5K ★★★★★

Pattern: Fixed sliding window | **Time:** $O(n)$ | **Space:** $O(1)$ **Problem:** Max vowels in substring of length k **Example:** $s = \text{"abciidef"}, k = 3 \rightarrow 3$

```
public int maxVowels(String s, int k) {
    Set<Character> vowels = new HashSet<>
(Arrays.asList('a', 'e', 'i', 'o', 'u'));
    int count = 0;
    for (int i = 0; i < k; i++) {
        if (vowels.contains(s.charAt(i))) count++;
    }
    int maxCount = count;

    for (int i = k; i < s.length(); i++) {
        if (vowels.contains(s.charAt(i - k))) count--;
        if (vowels.contains(s.charAt(i))) count++;
        maxCount = Math.max(maxCount, count);
    }
    return maxCount;
}
```

51. LC 994 - Rotting Oranges | 🍌 12K ★★★★★

Pattern: Multi-source BFS | **Time:** $O(m \times n)$ | **Space:** $O(m \times n)$ **Problem:** Minutes until all oranges rot (or -1 if impossible)**Example:** $\text{grid} = [[2,1,1],[1,1,0],[0,1,1]] \rightarrow 4$

```
public int orangesRotting(int[][] grid) {
    Queue<int[]> queue = new LinkedList<>();
    int fresh = 0;

    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == 2) queue.offer(new int[]{i, j});
            else if (grid[i][j] == 1) fresh++;
        }
    }

    if (fresh == 0) return 0;

    int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};
    int minutes = 0;
```

```

while (!queue.isEmpty()) {
    int size = queue.size();
    boolean rotted = false;
    for (int i = 0; i < size; i++) {
        int[] cell = queue.poll();
        for (int[] dir : dirs) {
            int r = cell[0] + dir[0], c = cell[1] + dir[1];
            if (r >= 0 && r < grid.length && c >= 0 && c <
grid[0].length &&
                grid[r][c] == 1) {
                grid[r][c] = 2;
                queue.offer(new int[]{r, c});
                fresh--;
                rotted = true;
            }
        }
    }
    if (rotted) minutes++;
}
return fresh == 0 ? minutes : -1;
}

```

52. LC 133 - Clone Graph | 👍 10K ★★★★★

Pattern: DFS with HashMap | **Time:** $O(V+E)$ | **Space:** $O(V)$

Problem: Deep copy of graph

Example: `adjList = [[2,4],[1,3],[2,4],[1,3]]` → cloned graph

```

private Map<Node, Node> visited = new HashMap<>();

public Node cloneGraph(Node node) {
    if (node == null) return null;
    if (visited.containsKey(node)) return visited.get(node);

    Node cloneNode = new Node(node.val, new ArrayList<>());
    visited.put(node, cloneNode);

    for (Node neighbor : node.neighbors) {
        cloneNode.neighbors.add(cloneGraph(neighbor));
    }
    return cloneNode;
}

```

53. LC 733 - Flood Fill | 👍 8K ★★★★★

Pattern: DFS/BFS | **Time:** $O(m \times n)$ | **Space:** $O(m \times n)$

Problem: Perform flood fill starting from (sr, sc)

Example: `image = [[1,1,1],[1,1,0],[1,0,1]]`, `sr=1`, `sc=1`, `newColor=2` → `[[2,2,2],[2,2,0],[2,0,1]]`

```
public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
    int original = image[sr][sc];
    if (original == newColor) return image;
    dfs(image, sr, sc, original, newColor);
    return image;
}

private void dfs(int[][] image, int r, int c, int original, int newColor) {
    if (r < 0 || r >= image.length || c < 0 || c >= image[0].length ||
        image[r][c] != original) return;
    image[r][c] = newColor;
    dfs(image, r+1, c, original, newColor);
    dfs(image, r-1, c, original, newColor);
    dfs(image, r, c+1, original, newColor);
    dfs(image, r, c-1, original, newColor);
}
```

54. LC 718 - Maximum Length of Repeated Subarray | 🍌 8K ⭐⭐⭐⭐

Pattern: 2D DP | **Time:** $O(m \times n)$ | **Space:** $O(m \times n)$

Problem: Length of longest common subarray

Example: `nums1 = [1,2,3,2,1]`, `nums2 = [3,2,1,4,7]` → 3 ([3,2,1])

```
public int findLength(int[] nums1, int[] nums2) {
    int m = nums1.length, n = nums2.length;
    int[][] dp = new int[m + 1][n + 1];
    int maxLen = 0;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (nums1[i-1] == nums2[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1;
                maxLen = Math.max(maxLen, dp[i][j]);
            }
        }
    }
    return maxLen;
}
```

55. LC 70 - Climbing Stairs | 👍 18K ★★★★★

Pattern: Fibonacci | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Ways to climb n stairs (1 or 2 steps at a time)

Example: $n = 3 \rightarrow 3$ (1+1+1, 1+2, 2+1)

```
public int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

56. LC 62 - Unique Paths | 👍 15K ★★★★★

Pattern: 2D grid DP | **Time:** $O(m \times n)$ | **Space:** $O(n)$

Problem: Unique paths from top-left to bottom-right (only right/down moves)

Example: $m = 3, n = 7 \rightarrow 28$

```
public int uniquePaths(int m, int n) {
    int[] dp = new int[n];
    Arrays.fill(dp, 1);
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j-1];
        }
    }
    return dp[n-1];
}
```

57. LC 416 - Partition Equal Subset Sum | 👍 12K ★★★★★

Pattern: 0/1 knapsack | **Time:** $O(n \times \text{sum})$ | **Space:** $O(\text{sum})$

Problem: Can array be partitioned into two equal sum subsets?

Example: $\text{nums} = [1, 5, 11, 5] \rightarrow \text{true}$ ([1,5,5] and [11])

```
public boolean canPartition(int[] nums) {
    int total = 0;
    for (int num : nums) total += num;
    if (total % 2 != 0) return false;

    int target = total / 2;
    boolean[] dp = new boolean[target + 1];
    dp[0] = true;

    for (int num : nums) {
        for (int j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j - num];
        }
    }
    return dp[target];
}
```

58. LC 295 - Find Median from Data Stream | 👍 14K ★★★★★

Pattern: Two heaps | **Time:** $O(\log n)$ add, $O(1)$ find | **Space:** $O(n)$

Problem: Design data structure to find median

Example: `addNum(1)`, `addNum(2)`, `findMedian()` → 1.5

```
class MedianFinder {
    PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a,b) -> b-a);
    PriorityQueue<Integer> minHeap = new PriorityQueue<>();

    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());
        if (maxHeap.size() < minHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
    }

    public double findMedian() {
        return maxHeap.size() > minHeap.size() ? maxHeap.peek() :
            (maxHeap.peek() + minHeap.peek()) / 2.0;
    }
}
```

59. LC 767 - Reorganize String | 👍 8K ★★★★★

Pattern: Max heap greedy | **Time:** $O(n \log k)$ | **Space:** $O(k)$

Problem: Rearrange so no two adjacent characters are the same

Example: $s = \text{"aab"} \rightarrow \text{"aba"}$

```
public String reorganizeString(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char c : s.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
        if (freq.get(c) > (s.length() + 1) / 2) return "";
    }

    PriorityQueue<Character> maxHeap = new PriorityQueue<>((a,b) ->
        freq.get(b) - freq.get(a));
    maxHeap.addAll(freq.keySet());

    StringBuilder result = new StringBuilder();
    while (maxHeap.size() > 1) {
        char first = maxHeap.poll();
        char second = maxHeap.poll();
        result.append(first).append(second);

        freq.put(first, freq.get(first) - 1);
        freq.put(second, freq.get(second) - 1);

        if (freq.get(first) > 0) maxHeap.offer(first);
        if (freq.get(second) > 0) maxHeap.offer(second);
    }

    if (!maxHeap.isEmpty()) result.append(maxHeap.poll());
    return result.toString();
}
```

60. LC 973 - K Closest Points to Origin | 🍌 10K ★★★★★

Pattern: Max heap of size k | **Time:** $O(n \log k)$ | **Space:** $O(k)$

Problem: Return k closest points to origin

Example: $\text{points} = [[1,3], [-2,2]], k = 1 \rightarrow [[-2,2]]$

```
public int[][] kClosest(int[][] points, int k) {
    PriorityQueue<int[]> maxHeap = new PriorityQueue<>((a,b) ->
        (b[0]*b[0] + b[1]*b[1]) - (a[0]*a[0] + a[1]*a[1]));

    for (int[] point : points) {
        maxHeap.offer(point);
        if (maxHeap.size() > k) {
            maxHeap.poll();
        }
    }
}
```

```

    }

    int[][] result = new int[k][2];
    for (int i = 0; i < k; i++) {
        result[i] = maxHeap.poll();
    }
    return result;
}

```

61. LC 102 - Binary Tree Level Order Traversal | 🍌 16K ★★★★★

Pattern: BFS with levels | **Time:** O(n) | **Space:** O(n)

Problem: Return level order traversal

Example: root = [3,9,20,null,null,15,7] → [[3],[9,20],[15,7]]

```

public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}

```

62. LC 98 - Validate Binary Search Tree | 🍌 18K ★★★★★

Pattern: Inorder traversal or recursive bounds | **Time:** O(n) | **Space:** O(h)

Problem: Check if tree is valid BST

Example: root = [2,1,3] → true

```
public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private boolean validate(TreeNode node, Integer min, Integer max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max != null && node.val >= max)) {
        return false;
    }
    return validate(node.left, min, node.val) && validate(node.right, node.val, max);
}
```

63. LC 105 - Construct Tree from Preorder and Inorder | 👍 12K ★★★★★

Pattern: Recursive divide | **Time:** O(n) | **Space:** O(n)

Problem: Build tree from preorder and inorder traversals

Example: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7] → tree

```
int preIndex = 0;
Map<Integer, Integer> inMap = new HashMap<>();

public TreeNode buildTree(int[] preorder, int[] inorder) {
    for (int i = 0; i < inorder.length; i++) {
        inMap.put(inorder[i], i);
    }
    return build(preorder, 0, preorder.length - 1);
}

private TreeNode build(int[] preorder, int left, int right) {
    if (left > right) return null;
    TreeNode root = new TreeNode(preorder[preIndex++]);
    int inIndex = inMap.get(root.val);
    root.left = build(preorder, left, inIndex - 1);
    root.right = build(preorder, inIndex + 1, right);
    return root;
}
```

64. LC 138 - Copy List with Random Pointer | 👍 14K ★★★★★

Pattern: HashMap or interweaving | **Time:** O(n) | **Space:** O(n)

Problem: Deep copy list with random pointers

Example: `head = [[7,null],[13,0],[11,4],[10,2],[1,0]]` → deep copy

```
public Node copyRandomList(Node head) {
    if (head == null) return null;
    Map<Node, Node> map = new HashMap<>();

    Node curr = head;
    while (curr != null) {
        map.put(curr, new Node(curr.val));
        curr = curr.next;
    }

    curr = head;
    while (curr != null) {
        map.get(curr).next = map.get(curr.next);
        map.get(curr).random = map.get(curr.random);
        curr = curr.next;
    }
    return map.get(head);
}
```

65. LC 141 - Linked List Cycle | 👍 16K ★★★★★

Pattern: Floyd's cycle detection | **Time:** $O(n)$ | **Space:** $O(1)$

Problem: Detect if linked list has a cycle

Example: `head = [3,2,0,-4]`, `pos = 1` → true

```
public boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

66. LC 20 - Valid Parentheses | 👍 20K ★★★★★

Pattern: Stack matching | **Time:** $O(n)$ | **Space:** $O(n)$

Problem: Check if parentheses are valid

Example: `s = "()[]{}"` → true

```
public boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    Map<Character, Character> map = new HashMap<>();
    map.put(')', '('); map.put(']', '['); map.put('}', '{');

    for (char c : s.toCharArray()) {
        if (map.containsKey(c)) {
            if (stack.isEmpty() || stack.pop() != map.get(c)) return false;
        } else {
            stack.push(c);
        }
    }
    return stack.isEmpty();
}
```

67. LC 394 - Decode String | 👍 12K ★★★★★

Pattern: Stack for nested structures | **Time:** O(n) | **Space:** O(n)

Problem: Decode encoded string

Example: s = "3[a]2[bc]" → "aaabcbcb"

```
public String decodeString(String s) {
    Stack<Integer> countStack = new Stack<>();
    Stack<StringBuilder> stringStack = new Stack<>();
    StringBuilder current = new StringBuilder();
    int k = 0;

    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) {
            k = k * 10 + (c - '0');
        } else if (c == '[') {
            countStack.push(k);
            stringStack.push(current);
            current = new StringBuilder();
            k = 0;
        } else if (c == ']') {
            StringBuilder decoded = stringStack.pop();
            int repeat = countStack.pop();
            for (int i = 0; i < repeat; i++) {
                decoded.append(current);
            }
            current = decoded;
        } else {
            current.append(c);
        }
    }
}
```

```
        return current.toString();  
    }
```

68. LC 155 - Min Stack | 👍 15K ★★★★★

Pattern: Two stacks | **Time:** $O(1)$ | **Space:** $O(n)$

Problem: Stack with $O(1)$ getMin

Example: `MinStack stack = new MinStack(); stack.push(-2); stack.getMin() → -2`

```
class MinStack {  
    Stack<Integer> stack = new Stack<>();  
    Stack<Integer> minStack = new Stack<>();  
  
    public void push(int val) {  
        stack.push(val);  
        if (minStack.isEmpty() || val <= minStack.peek()) {  
            minStack.push(val);  
        }  
    }  
  
    public void pop() {  
        if (stack.pop().equals(minStack.peek())) {  
            minStack.pop();  
        }  
    }  
  
    public int top() {  
        return stack.peek();  
    }  
  
    public int getMin() {  
        return minStack.peek();  
    }  
}
```

69. LC 387 - First Non-Repeating Character in Stream | 👍 5K ★★★★★

Pattern: Queue + HashMap | **Time:** $O(1)$ per operation | **Space:** $O(n)$

Problem: Design data structure to find first non-repeating character in stream

Example:

```

add('a') → 'a'
add('a') → null (both 'a' are repeating)
add('b') → 'b'
add('c') → 'b'

```

```

class FirstUnique {
    Queue<Integer> queue = new LinkedList<>();
    Map<Integer, Integer> freq = new HashMap<>();

    public void add(int num) {
        freq.put(num, freq.getOrDefault(num, 0) + 1);
        queue.offer(num);

        // Remove repeating numbers from front
        while (!queue.isEmpty() && freq.get(queue.peek()) > 1) {
            queue.poll();
        }
    }

    public int firstUnique() {
        return queue.isEmpty() ? -1 : queue.peek();
    }
}

```

70. LC 244 - Shortest Word Distance II | 👍 4K ★★★★★

Pattern: Preprocessing + two pointers | **Time:** O(m+n) per query | **Space:** O(N)

Problem: Design class to find shortest distance between two words, called multiple times

Example:

```

words = ["practice", "makes", "perfect", "coding", "makes"]
shortest("coding", "practice") → 3
shortest("makes", "coding") → 1

```

```

class WordDistance {
    Map<String, List<Integer>> map = new HashMap<>();

    public WordDistance(String[] words) {
        for (int i = 0; i < words.length; i++) {
            map.putIfAbsent(words[i], new ArrayList<>());
            map.get(words[i]).add(i);
        }
    }
}

```

```
}

public int shortest(String word1, String word2) {
    List<Integer> list1 = map.get(word1);
    List<Integer> list2 = map.get(word2);
    int minDist = Integer.MAX_VALUE;
    int i = 0, j = 0;

    // Two pointers on two sorted lists
    while (i < list1.size() && j < list2.size()) {
        int idx1 = list1.get(i);
        int idx2 = list2.get(j);
        minDist = Math.min(minDist, Math.abs(idx1 - idx2));

        if (idx1 < idx2) {
            i++;
        } else {
            j++;
        }
    }
    return minDist;
}
```

IMPORTANT FOLLOW-UP QUESTIONS

Arrays & Hashing

LC 56 - Merge Intervals

- Q: What if intervals are already sorted?
- A: Skip sorting, directly merge. Time becomes $O(n)$.

LC 347 - Top K Frequent Elements

- Q: What if we need top k from a stream?
- A: Use Min Heap of size k. Update frequency map and rebuild/replace if needed.

LC 560 - Subarray Sum Equals K

- Q: What if array has only positive numbers?
- A: Can use sliding window (two pointers) - $O(n)$ time, $O(1)$ space.

LC 238 - Product of Array Except Self

- Q: What if we can use division?
- A: Calculate total product, divide by `nums[i]`. Handle zero case: count zeros, if >1 all zeros, if $=1$ only that position is non-zero.

LC 128 - Longest Consecutive Sequence

- Q: Find longest sequence with specific difference d (not just 1)?
- A: Similar HashSet approach, check $num-d$ and count $num+d$.

Sliding Window

LC 3 - Longest Substring Without Repeating Characters

- Q: At most K distinct characters allowed?
- A: Similar sliding window, shrink when distinct count $> k$. Very common variation.

LC 76 - Minimum Window Substring

- Q: Return all minimum windows if multiple exist?
- A: Store all windows with same minimum length in a list.

Two Pointers

LC 15 - 3Sum

- Q: Find all quadruplets that sum to target (4Sum)?
- A: Add one more outer loop. Time becomes $O(n^3)$. LC 18.

LC 42 - Trapping Rain Water

- Q: 2D version (rain water II)?
- A: Use priority queue (min heap) starting from borders. LC 407. Very famous extension.

LC 287 - Find the Duplicate Number

- Q: Find the cycle start (why Floyd's cycle works)?
- A: Array acts as linked list: value at index is next pointer. Duplicate creates cycle.

Graphs & BFS/DFS

LC 200 - Number of Islands

- Q: Dynamic grid - islands added/removed online?
- A: Union-Find with rank/path compression. LC 305. Very important for system design.

LC 207 - Course Schedule

- Q: Return the actual course order?
- A: Same Kahn's algorithm, store courses as you process. LC 210. Very common follow-up.

LC 127 - Word Ladder

- Q: Return all shortest transformation sequences?
- A: Bidirectional BFS + backtracking to reconstruct paths. LC 126.

Dynamic Programming

LC 322 - Coin Change

- Q: Count number of ways to make amount (not minimum coins)?
- A: Similar DP but $dp[i] += dp[i-coin]$ instead of min. LC 518. Very common.

LC 139 - Word Break

- Q: Return all possible word break sentences?
- A: DFS with memoization. LC 140. Common extension.

LC 300 - Longest Increasing Subsequence

- Q: Return the actual sequence, not just length?
- A: Track parent pointers during DP, reconstruct path at end.

LC 53 - Maximum Subarray (Kadane's Algorithm)

- Q: Return the actual subarray (start and end indices)?
- A: Track start/end indices when updating maxSum. Very common ask.

LC 198 - House Robber

- Q: Houses in a circle (first and last are adjacent)?
- A: Rob twice: exclude first house, exclude last house. Take max. LC 213. Very common.

LC 416 - Partition Equal Subset Sum

- Q: Can you partition into k equal sum subsets?
- A: Backtracking with pruning. LC 698. More general version.

LC 55 - Jump Game

- Q: Return minimum number of jumps?
- A: Already covered in LC 45 - greedy BFS approach.

Heap & Priority Queue

LC 215 - Kth Largest Element

- Q: Find kth largest in a data stream?
- A: Maintain min heap of size k continuously. LC 703.

LC 295 - Find Median from Data Stream

- Q: Find median of last k elements (sliding window median)?
- A: Two heaps with removal operation or use multiset/TreeMap.

LC 23 - Merge K Sorted Lists

- Q: What if lists are too large to fit in memory?
- A: External merge sort - process in chunks, stream from disk.

Trees & BST

LC 236 - Lowest Common Ancestor (Binary Tree)

- Q: What if it's a BST instead?
- A: Use BST property: if both nodes < root, go left; if both > root, go right. $O(h)$ time, $O(1)$ space. LC 235.

LC 98 - Validate Binary Search Tree

- Q: Find kth smallest element in BST?
- A: Inorder traversal with counter. LC 230. Very common.

LC 124 - Binary Tree Maximum Path Sum

- Q: Path must go through root?
- A: Simpler - just $\text{left_gain} + \text{root.val} + \text{right_gain}$. No need for global max.

Linked Lists

LC 146 - LRU Cache

- Q: Implement LFU (Least Frequently Used) instead?
- A: Min heap + frequency map. LC 460. Very common system design question.

LC 206 - Reverse Linked List

- Q: Reverse only between positions left and right?
- A: Find position, reverse segment, reconnect. LC 92.

LC 141 - Linked List Cycle

- Q: Find the start node of the cycle?
- A: After detecting cycle with slow/fast, reset slow to head, move both one step until they meet. LC 142. Very common.

Stack & Monotonic Stack

LC 84 - Largest Rectangle in Histogram

- Q: Maximal rectangle in 2D binary matrix?
- A: Treat each row as histogram base. LC 85. Famous extension.

LC 739 - Daily Temperatures

- Q: Previous warmer day instead of next?
- A: Iterate from right to left, maintain decreasing stack.

Backtracking

LC 78 - Subsets

- Q: Input array has duplicates?
- A: Sort first, skip duplicates at same recursion level. LC 90.

LC 46 - Permutations

- Q: Input has duplicate numbers?
- A: Sort + skip duplicates at same level. LC 47. Very common.

LC 79 - Word Search

- Q: Find all words from a dictionary in the board?
- A: Build Trie from dictionary, DFS with Trie. LC 212. Very important optimization.

Total: 69 problems with 35+ essential follow-ups Complete coverage of all requested questions
Print-ready for interviews | ~50 pages landscape

Good luck! 🚀

Total: 70 problems sorted by interview priority Print-ready | ~50-60 pages landscape

Study Strategy:

- Week 4-3: Tier 1 (Problems 1-12)
- Week 2-1: Tier 2 (Problems 13-36)
- Week 0: Review Tier 3, Pattern Guide

Good luck! 🚀